

Project Report

REITttersel: Equity Risk Solver for S-REITs

Intelligent Reasoning Systems

Student Name	Student ID
Jason Tay Neng Wei	A0265092A

Table of Contents

- 1. Executive Summary
- 2. Business Case / Market Research
 - 2.1. Business Case
 - 2.2. Competitive Positioning
 - 2.3. Literature Review
- 3. System Design / Model
 - 3.1. Original Design
 - 3.2. Data Definitions
 - 3.2.1. Labels / Ground Truth
 - 3.2.2. Theoretical Benchmark
 - 3.2.3. Threshold-Based Label Engineering Rationale
 - 3.3. XGBoost / Macro Data Design
 - 3.4. Mamdani Fuzzy Rules / Micro Data Design
 - 3.5. Full Pipeline / Hybrid Model
 - 3.6. UI Prototype
- 4. System Development & Implementation
 - 4.1. Early Development / Dead Forks
 - 4.2. Data Scraping and Data Engineering
 - 4.2.1. Threshold-Based Annual Label Engineering
 - 4.3. XGBoost Development
 - 4.3.1. Multi-Configuration Controlled Experiment Design
 - 4.3.2. Hyperparameter Search Strategy
 - 4.4. Pipeline and Application Delivery
- 5. Findings and Discussion
 - 5.1. Evaluation for XGBoost Best Version
 - 5.2. Evaluation for XGBoost Historical Versions
 - 5.3. Evaluation for Mamdani Layer and Full Pipeline
 - 5.4. Representation Tables and Graphs
 - 5.5. Optuna and DEAP as an Adversarial Error-Surfacing System
 - 5.6. Developed Models and Final Interpretation
- 6. Future Work
 - 6.1. Conclusion and Afterthoughts
- 7. References
- Appendix A. Project Proposal
- Appendix B. Mapped System Functionalities against MR, RS, CGS Modules
 - Appendix B.1. Decision Automation
 - Appendix B.2. Business Resource Optimization / Evolutionary Computing
 - Appendix B.3. Knowledge Discovery and Data Mining
 - Appendix B.4. Cognitive Techniques / Tools
- Appendix C. Installation and User Guide
 - Appendix C.1. Docker submission / demo mode
- Appendix D. Data Sources

1. Executive Summary

This project develops a hybrid distress-monitoring system for Singapore REITs that combines an annual Mamdani fuzzy reasoning layer, a short-horizon XGBoost macro overlay, and a cumulative abnormal return path overlay. The annual layer provides the structural accounting anchor, while the macro and market overlays make the runtime score more responsive when refinancing conditions or post-filing market reactions deteriorate. Evaluation results show that Mamdani is the strongest standalone annual classifier, while the full hybrid score is more aggressive in surfacing distressed names and performs better as a continuous runtime prioritization signal.

2. Business Case / Market Research

2.1. Business Case

Singapore REITs (S-REITs) are income-oriented instruments that are highly exposed to financing costs, refinancing structure, and macroeconomic variables, on top of their internal accounting health statistics.

Recent sector stress is especially relevant because S-REIT balance sheets are sensitive to higher debt costs and refinancing walls. There are known recurrent risks such as debt-cost sensitivity, refinancing risk, and valuation compression.

For example, geopolitical stress events such as rate regime changes and black swan events (COVID-19, Operation Epic Fury 2026) can propagate into downstream distress for S-REITs, as was demonstrated with notable S-REIT distresses such as Lippo Malls Indonesia Retail Trust (D5IU) and Prime US REIT (OXMU).

Under the revised MAS framework, a REIT with $ICR < 1.5x$ can be blocked from taking on additional debt even if its aggregate leverage is still low. In practical terms, a REIT may appear lightly geared on paper yet still face funding stress if weaker earnings push interest coverage below the regulatory threshold.

For example, a REIT with 20% gearing but $ICR = 1.2x$ may still be unable to borrow for recovery or refinancing needs, leading to shareholder dilution or price impact.

2.2. Competitive Positioning

There are a variety of public, simple yield screens when selecting S-REITs available to Singapore retail investors.

- **REITsavvy Screener**, which exposes filters and raw metrics but leaves reasoning to the user.
- **Fifth Person**, which presents live S-REIT data for general reference but does not encode rule-based distress scoring.
- **DBS Research InsightsDirect**, which contains analyst reasoning but is episodic, institutional, and not transparently rerunnable by a retail user.

Existing tools either display metrics without reasoning (REITsavvy, Fifth Person) or apply reasoning episodically behind institutional paywalls (DBS Research). There is a lack of quantitative evaluation available that captures the safety of current S-REITs, or whether current distributions remain sustainable under changing policy and inflation.

This project addresses that gap by building intelligent reasoning systems that combine macro regime signals with REIT-level financial health indicators.

2.3. Literature Review

The existing literature provided several design lessons that were directly useful. Shumway showed that distress prediction improves when the model is kept strictly time-ordered and observable at the point of prediction, rather than built from static snapshots that blur timing discipline [1, Sec. III, p. 111; Sec. IV.B, p. 115; Sec. V, pp. 116-117]. Campbell extended that logic by combining slower accounting anchors with faster-moving market variables, including lagged return paths and volatility, so that risk can still update

between reporting dates [2, Sec. 2, pp. 6-7; Sec. 3, pp. 10-12; Appendix, pp. 28-29]. Martyushev et al. reinforced the same broad lesson in a more recent machine-learning setting: temporal structure, engineered lag features, and disciplined holdout evaluation materially improve predictive usefulness [4, Sec. 3.4; Sec. 3.7; Sec. 4.2].

The literature also supported keeping the output interpretable rather than purely predictive. Cheng, Su, and Li treated financial distress as a graded state through fuzzy modelling instead of forcing a hard binary boundary, while Campbell showed that risk should be allowed to move when new market information arrives even if the accounting base is unchanged [3, Sec. "Fuzzy Regression Model," p. 87; Eq. 12, pp. 83-84] [2, Sec. 2, p. 7; Sec. 3, p. 10, Eq. (1)]. Those two ideas are reflected in this project's intended design and final structure: an annual Mamdani reasoning layer provides the stable accounting anchor, while the macro and Cumulative Abnormal Returns (CAR)-path overlays update the score when conditions change before the next annual filing.

At the same time, the existing literature left several gaps that this project chose to address. The papers written by Shumway, Campbell, and Cheng were developed for general corporate distress settings rather than the specific constraints of S-REITs, where leverage caps, refinancing pressure, mandatory distributions, and MAS coverage rules can become binding well before formal insolvency [1, Sec. 1, pp. 101-104] [2, Sec. 1, pp. 1-5] [3, Sec. "Research Restrictions," p. 84]. Cheng also acknowledged that a ratio-only framework omits broader business-cycle and non-financial effects [3, Sec. "Research Restrictions," p. 84]. This project addresses that gap by combining REIT-specific financial ratios with a macro rate-stress layer and explicit MAS-linked reasoning instead of treating distress as solely a firm-failure problem.

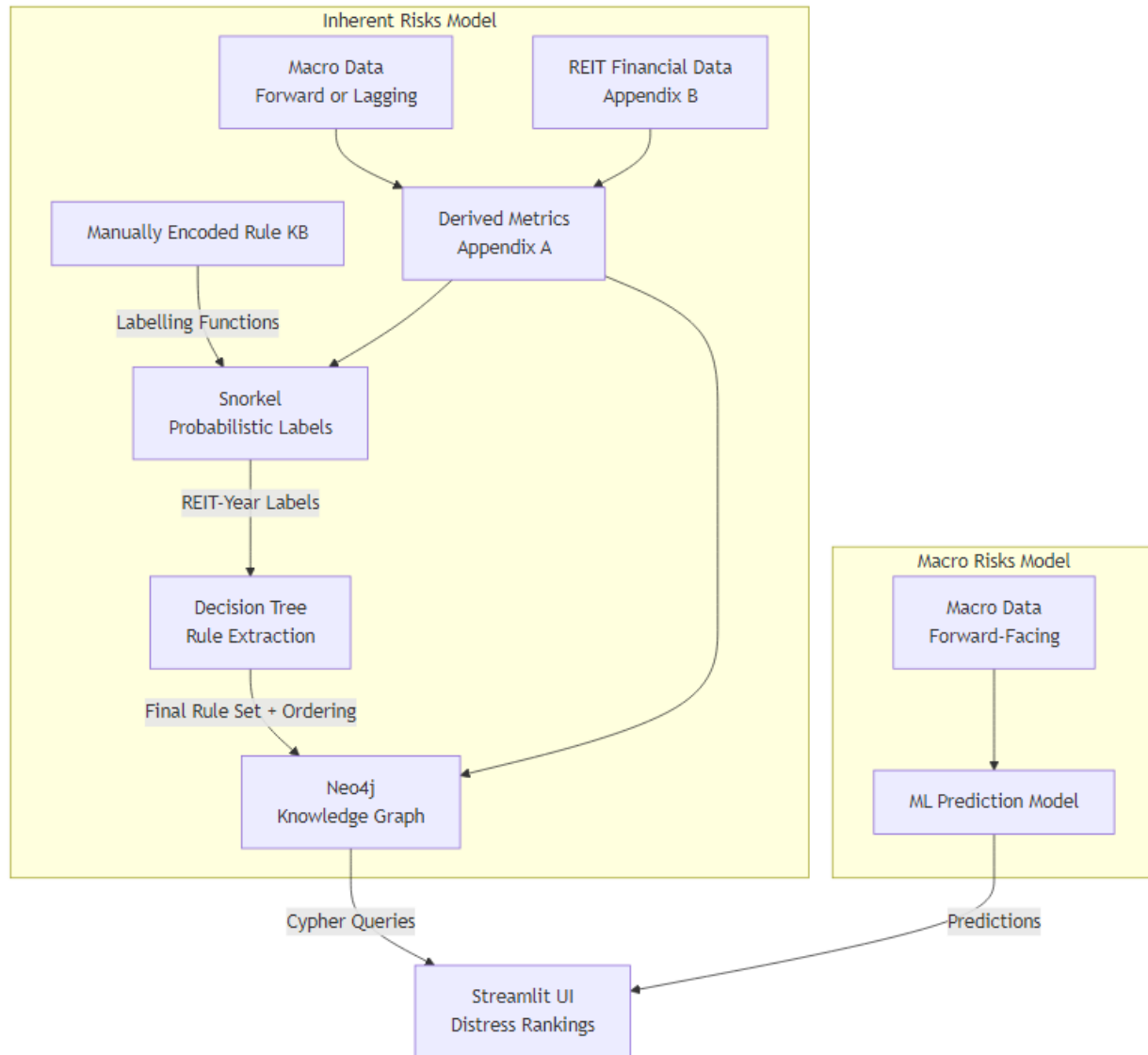
Another gap was architectural. The prior papers mainly used single-model prediction pipelines, whereas this project separates the problem into three linked parts: an annual fuzzy anchor from accounting fundamentals, a macro overlay "watchdog" from forward rate conditions, and a market-reaction overlay from cumulative abnormal returns. That design choice is closest in spirit to Campbell's multi-speed view of risk and to Cheng's emphasis on explainable graded outputs, but it goes further by turning those ideas into a hybrid decision-support system tailored to S-REIT monitoring instead of a single standalone prediction model [2, Sec. 2, pp. 6-7; Sec. 3, pp. 10-12] [3, Sec. "Results," pp. 84-90] [4, Sec. 3.6.4; Sec. 3.7; Sec. 4.2].

3. System Design / Model

3.1. Original Design

The original proposal described a hybrid reasoning architecture that combined induced rules from Snorkel weak supervision [5], Orange/Decision Tree rule extraction, Neo4j storage, and a Streamlit front end. The design emphasized explainability first, with the macro model acting as an additional overlay rather than replacing the reasoning layer.

The original design architecture from the proposal is reproduced below because it is still useful for showing the intended starting point:



Although the final design incorporates the bulk of this architecture, Snorkel and decision-tree induction were removed during implementation due to the additional overhead that using Snorkel’s voting LF functions entailed. The current implementation now relies on threshold-derived labels and a seeded Mamdani fuzzy rule system evaluated in Python. The final architecture is therefore best described as a design evolution rather than a straight proposal-to-code translation.

3.2. Data Definitions

3.2.1. Labels / Ground Truth

The project does not use hand-labelled distress classes as its ground truth. Instead, it derives the annual label from how each REIT performs against the sector after its fiscal-year-end filing anchor.

For each ticker-period, the pipeline takes the first trading day on or after the fiscal year end, then compounds forward abnormal returns over 63 and 126 trading days. Here, abnormal return means:

$\text{REIT daily return} - \text{SGX iEdge REIT index daily return}$

The main label used in the project is `label_126wd`, which is derived from `car_126wd` and stored in

`reit_labels.fact_distress_label` together with the anchor date, forward-window end dates, and data-quality counts such as `null_count` and `non_ok_count`.

In practical terms, this means the system treats market reaction after the annual anchor as the closest available proxy for distress. Rather than asking whether the REIT price fell in isolation, it asks whether the REIT materially underperformed or outperformed the S-REIT benchmark over the following half-year window. That makes the label more suitable for this project than a raw price move or an informal manual class.

3.2.2. Theoretical Benchmark

The benchmark in this project is not a single external label set. Instead, each part of the system is judged against the role it is supposed to play.

For the annual distress layer, the practical benchmark is whether the model's classification agrees with `label_126wd`, which is the project's own ground-truth label derived from forward 126-trading-day abnormal returns. In other words, the annual reasoning layer is evaluated by asking a simple question: does its distress judgement line up with the later relative market outcome for that REIT?

For the macro layer, the benchmark is narrower. The XGBoost model is not trying to predict firm distress directly. It is trying to predict short-horizon SORA movement well enough to act as a useful rate-stress overlay. That is why its performance is assessed through holdout forecasting results rather than by comparing it straight to `label_126wd`.

The final evaluation then brings the system views together and compares four levels of interpretation: a simple baseline, the annual Mamdani score, a REFI-only stress proxy, and the full hybrid score. This matters because it shows whether the added complexity is actually useful. If the hybrid score cannot outperform the simpler views, then the macro and CAR-path overlays are not earning their place in the design.

3.2.3. Threshold-Based Label Engineering Rationale

The label thresholds are intentionally simple and conservative. If `car_126wd` < -15%, the REIT is labelled `DISTRESSED`. If `car_126wd` > +5%, it is labelled `HEALTHY`. Everything in between is labelled `WATCH`.

This creates an intentionally asymmetric rule. A REIT is only called distressed when it materially underperforms the sector benchmark over the next 126 trading days, while a smaller positive threshold is enough to classify it as healthy. The middle range is left as `WATCH` because the market signal is not decisive enough to justify a stronger conclusion.

This matters because the project is trying to detect meaningful post-filing deterioration and ignore ordinary price noise. A wider negative threshold helps avoid calling every weak period a distress event, while the middle bucket preserves uncertainty instead of forcing borderline cases into a false healthy-or-distressed split.

In plain language, the label says that `DISTRESSED` means the REIT performed clearly worse than the sector after the filing anchor, `HEALTHY` means it held up or outperformed clearly, and `WATCH` means the evidence was mixed.

3.3. XGBoost / Macro Data Design

The macro model used in the final app belongs to what the project informally calls the P family, meaning Parquet-direct. In this family, features are built directly from the large parquet-based macro dump, especially forward-looking rate-expectation fields such as `expected_bps`, and the model is trained to predict short-horizon SORA movement. The macro model is not meant to predict REIT distress directly. Its job is narrower: estimate short-horizon movement in SORA so the app can translate rate stress into a refinancing-risk overlay for each REIT. In the deployed setup, the active target is `option2_change`, which predicts the future change in SORA over the next 10 SGX trading days rather than the absolute rate level.

This design choice matters because the project is trying to detect changes in the rate environment, and not simply restate the current regime. That is why the model gives weight to market-expectation variables such as `expected_bps`, `p_no_change`, `margin_over_second`, and `days_to_next_fomc`, while also using local term-structure signals such as `sora_term_spread_t2`, `expected_bps_minus_sora_90d`, and `sora_curve_steepness`. In plain terms, the model asks both what the market expects the Fed and rate path to do next, and what the recent local SORA curve is already implying.

The feature design also shows that the model is intentionally path-aware rather than snapshot-only. It uses lag differences, realized volatility, drawdown from recent peak, distance from moving average, and short-horizon acceleration to describe how SORA has been moving recently. This is consistent with the narrower lesson highlighted in Martyushev et al.: temporal path and trend information can be more predictive than a single point-in-time reading. The caveat is that the Martyushev paper expresses that idea through STL-derived trend components [4, Figs. 2-3, Sec. 3.4] while `REITterrastel` uses manual lag and momentum features instead.

The script also makes a deliberate effort to avoid leakage and overfitting. For the active change target, it excludes `sora_level_t2` and `sora_3m_t2` because those level variables carry heavy regime information that can distort a change forecast. This is also consistent with the feature-exclusion discipline highlighted in Martyushev et al., where train-test separation and protection against near-circular inputs are treated as part of credible model design rather than as optional cleanup [4, Sec. 3.3]. The train-test split is strictly time ordered, with 70% for training, 20% for testing, and a 63-row gap in between. That gap is important because it reduces overlap (“bleeding”) between feature history and forward target windows, making the holdout evaluation more credible for a time-series setting.

3.4. Mamdani Fuzzy Rules / Micro Data Design

The Mamdani layer is the project’s annual reasoning core. Its job is to take a REIT’s annual fundamentals and turn them into a graded distress score that is easier to interpret than a single hard threshold. In practice, it answers a question like: given this combination of coverage, leverage, refinancing pressure, and payout strain, does the REIT look stable, worth watching, already high risk, or close to critical?

The chosen inputs follow a deliberate hierarchy. `ICR` and `GEARING` are the strongest rule candidates because they have clear regulatory and covenant anchors. `PAYOUT_RATIO`, `DSCR`, and `REFI_RISK` are the next tier because they capture REIT-specific structural stress: whether operations can service debt, whether distributions are still supported by cash generation, and whether too much debt is concentrated in the near term. `NET_DEBT_EBITDA` and `FFO_COVERAGE` are weaker standalone anchors, so they work better as corroborating evidence than as the main basis of the system. The layer also includes `NULL_COUNT`, which is not a normal financial ratio but a derived meta-risk input built from annual missing-input status. By design, `non_ok_count` is kept separately for diagnostics and display, but it is not currently a Mamdani rule input.

The rule design reflects that hierarchy. Some rules are direct alarms because the underlying metric already has a principled threshold, such as very weak `ICR`, very weak `DSCR`, or extreme `REFI_RISK`. Other rules are confirmation or combination rules, such as weak coverage together with stretched leverage, or over-distribution together with `FFO` shortfall.

This matters because the model is not trying to fuzzify every available metric. It is restricting strong rules to variables with defensible REIT-domain anchors, then using supporting variables to confirm or amplify the signal when multiple weaknesses line up.

The implementation also checks whether each ratio is trustworthy before letting it influence the score at full strength. This matters because some ratios stop behaving like normal business ratios during distress. When the underlying profit or cash base turns negative, or when the denominator becomes too small, the raw ratio can flip sign, explode in magnitude, or otherwise lose smooth business meaning.

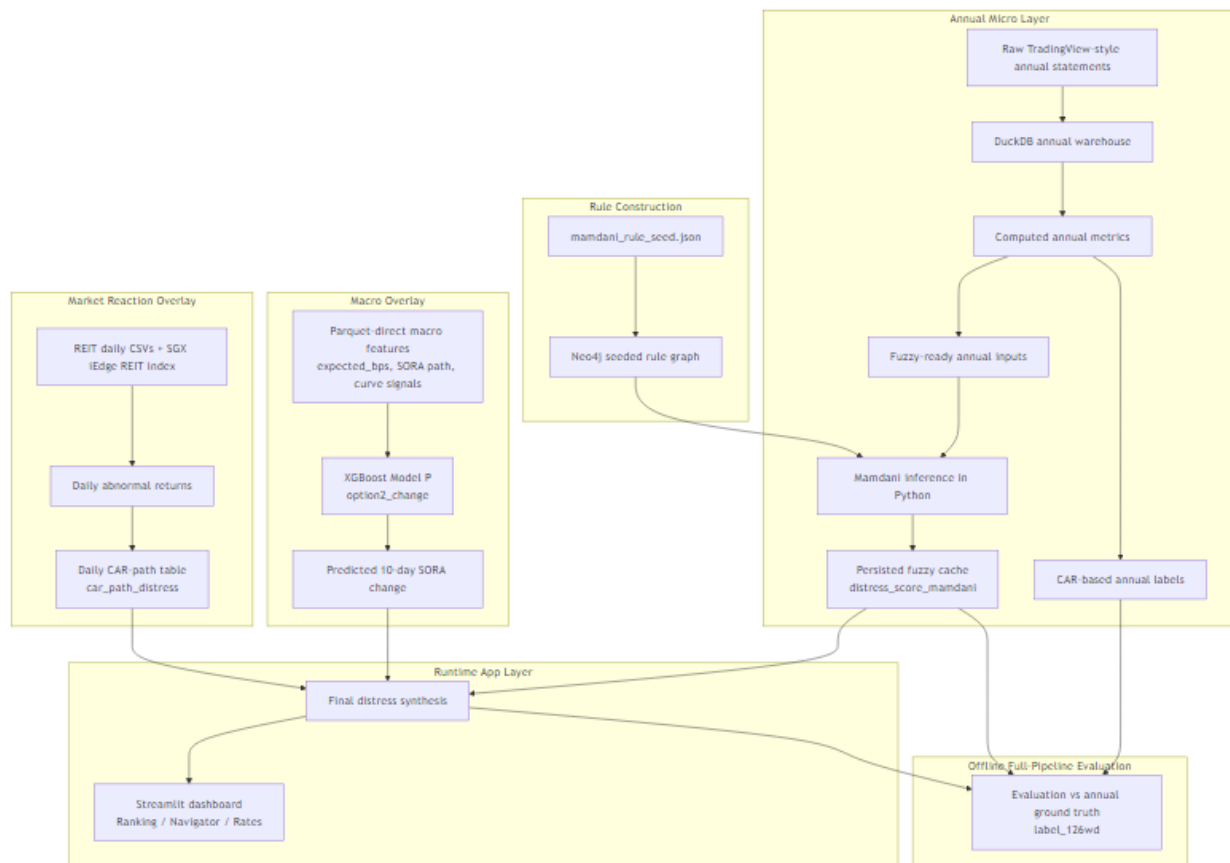
If a ratio is flagged as `NEGATIVE_BASE` or `DISTRESS_BASE`, the code therefore forces a distress-style interpretation because the accounting base itself is already broken. If the value is `PARTIAL`, `CLIPPED_SOURCE_SHARE`, or `LOW_DENOMINATOR`, the model reduces confidence instead of pretending the ratio is fully reliable. The final

Mamdani score is then blended back toward neutral when rule activation is weak, which makes the annual signal more cautious when the underlying evidence is incomplete or unstable. Just as importantly, these fuzzy-ready interpretations are kept separate from the raw warehouse metrics rather than overwriting them, so the annual source data remains auditable.

3.5. Full Pipeline / Hybrid Model

The cleanest way to understand the full design is as an annual anchor with two faster-moving overlays. The annual anchor is the Mamdani score built from annual fundamentals. It represents the slower-moving balance-sheet view of distress and stays frozen until the next annual checkpoint. This is the stable base of the system because accounting weakness, leverage strain, and payout pressure do not usually reset every day.

The implemented architecture is shown below.



That annual anchor is then deliberately supplemented by the earlier XGBoost and CAR-path work, because annual fundamentals alone are “frozen” and incapable of explaining how risk changes between filing dates. The XGBoost layer contributes a macro rate-stress view by predicting short-horizon SORA change. This matters because refinancing pressure is not only a function of how much debt a REIT has, but also of what the rate environment is becoming. The design therefore uses the XGBoost output as a macro shock rather than as a standalone distress classifier.

The second overlay is the daily CAR-path layer. This uses cumulative abnormal return from the same annual filing anchor to show how the market has reacted since that disclosure point. Its role is different from the macro model. The macro layer captures a broad rate regime shift that can affect the sector, while the CAR-path layer captures the REIT-specific market path after the annual anchor. In other words, the macro layer asks whether funding conditions are worsening, while the CAR-path layer asks whether this particular REIT

is already trading like a weaker name.

This CAR-path layer is built from a separate daily table anchored to the same annual filing date. The anchor trading day starts at `accum_car_to_date = 0.0`, then abnormal returns are compounded forward day by day and mapped into a continuous `car_path_distress` score using the same -15% / +5% logic as the annual label scheme. It is important to note that this layer is not part of Mamdani rule firing. It is a separate runtime overlay.

The runtime `final_distress` score is the point where those three ideas are combined. The code starts from the frozen `distress_score_mamdani`, then adds a macro adjustment from `distress_sora` and a market-path adjustment from `car_path_distress`. `REFI_RISK` is the bridge between the annual and macro layers: a REIT with higher refinancing concentration is made more sensitive to the same macro rate shock than a REIT with lower near-term refinancing pressure. The CAR-path overlay is also given a neutral dead zone so that very small daily path moves do not cause the system to overreact too early.

This relationship is the real logic of the hybrid model. Mamdani answers the annual structural question, XG-Boost answers the short-horizon rate-regime question, and CAR path answers the market-reaction question.

The final design in `Design_v1a.txt` is therefore not three unrelated components placed side by side. It is a layered system in which annual accounting risk is treated as the base state, then adjusted at runtime by macro stress and by REIT-specific market behaviour between annual reporting dates. At runtime, the app reads the persisted annual outputs and rule traces from DuckDB rather than querying Neo4j for rule definitions live, which keeps the serving layer simpler and more reproducible.

3.6. UI Prototype

The UI was designed to make the hybrid model readable to a non-technical user. In practical terms, the interface is not just there to display a final distress score. Its job is to show how that score was formed, what annual anchor it came from, and how the macro and market-path overlays changed it at runtime.

The repository contains both design assets and a working implementation. The design references remain in `Common\Frontend\DesignDoc\Reitteratsel.pdf`, `Common\Frontend\DesignDoc\figma.png`, and the branding files such as `Reiterratsel_Wordmark.svg`, while the self-contained runnable submission copy is now served from `SystemCode\Frontend\reitteratsel_app.py`.

The current Streamlit interface exposes three pages: Ranking, Individual REIT Navigator, and Time Series (Rates). These pages reflect the same three-part logic used in the system design. The Ranking page gives a sector-wide view of current runtime distress scores, the Individual REIT Navigator breaks down one selected REIT in more detail, and the Rates page shows the macro side of the system through predicted versus actual rate behaviour.

The app resolves a user-selected simulation date, preserves REIT selection across page navigation, and attaches explanatory help text to many model-derived fields so the user can trace where each displayed value comes from. It also exposes the annual Mamdani base, the macro overlay, and the CAR-path contribution separately, which makes the final score easier to interpret and challenge. The current explanation layer is a persisted text trace of the top fired rules rather than a full interactive rule-strength visualizer. For proof-of-concept purposes, the app can also surface forward-looking fields such as `car_63wd`, `car_126wd`, and `label_126wd`.

Visually, the interface follows a dashboard-style layout with custom branding and a clear information hierarchy. That visual choice is consistent with the broader aim of the project: the UI should help the user understand why a REIT is being flagged, rather than simply presenting a score with no reasoning attached.

4. System Development & Implementation

4.1. Early Development / Dead Forks

The original proposal explored a weak-supervision route using Snorkel, followed by Orange or decision-tree rule extraction. This was a reasonable starting point because the annual S-REIT dataset is small and does not come with a ready-made distress label. At an early stage, weak supervision appeared to offer a way to construct labels first and derive interpretable rules afterward.

That direction was eventually abandoned because the weak-supervision branch became difficult to maintain in a clean way. It required repeated threshold design inside the labeling functions, further adjustments to stabilise vote behaviour, and ongoing checking of confidence patterns before it could produce usable pseudo-labels. In practical terms, the method was demanding substantial effort just to maintain the intermediate label-generation stage.

Orange and decision-tree extraction did not resolve that problem. Both methods still depended on Snorkel-generated pseudo-labels rather than on the final distress target itself. This meant that the extracted rules were not primary explanations of REIT distress, but second-stage approximations of an already hand-engineered weak-labelling layer. That made them less suitable as the final reasoning core of the project.

There was also an interpretability issue. The final system needed to justify clearly why a REIT should be treated as stable, watch, high risk, or critical. The weak-labelling and rule-extraction route added extra transformation steps, more tuning decisions, and more opportunities for unstable or awkward rule outputs, without giving a corresponding improvement in explanatory value.

The project therefore moved to a more direct design. Annual labels are now engineered directly from forward CAR relative to the S-REIT benchmark. The reasoning layer is implemented as a seeded Mamdani fuzzy system with explicit inputs, thresholds, and rule logic, as mentioned previously. The remaining part of the original design is that resulting annual labels and fuzzy outputs still persist to Neo4j as well as a DuckDB caching layer for runtime reuse.

This change simplified the logic of the system considerably. Instead of depending on proxy labels, extracted rules, and post-hoc interpretation, the final implementation uses a direct label definition and an explicit auditable rule base. That final design is much better aligned with the project's core goal of producing an explainable REIT distress-monitoring system.

4.2. Data Scraping and Data Engineering

The data-engineering pipeline is easier to understand if it is separated into two parts: the annual REIT fundamentals pipeline and the macro time-series pipeline. The two pipelines are built differently because they solve different data problems, and both sides include dedicated scraping or source-probing stages before downstream modelling begins.

On the REIT side, the project starts from TradingView HTML annual financial statements. The supporting scripts for this stage are spread across `Common\Micro\1_TradingView_Exploration`, `Common\Micro\2_HTML_Dumping`, and `Common\Micro\3_Serialize_Dump_To_CSV_Parquet`. In practical terms, the first folder contains exploratory helpers for navigating the TradingView financial tabs, the second contains the HTML dumping workflow itself, and the third converts the dumped HTML into structured warehouse outputs. Those statements are therefore first captured as raw source material, then converted into a structured row-based format. The serializer script does not simply dump text into a flat file; it maps statement labels into a reusable schema, assigns row identifiers, and writes the result into both DuckDB and per-ticker parquet files. This stage turns messy statement exports into a queryable warehouse with consistent row structure across REITs and years.

The next step is metric engineering. `build_reit_metrics.py` reads the consolidated annual warehouse and derives the project's usable REIT indicators from it. This is where ratios such as leverage, debt-service coverage, payout strain, and refinancing-related measures are computed from the raw annual statements.

The metric layer also joins in external series where needed, such as SORA 3M on or before the fiscal year end. This allows the final project to reason on a cleaned annual metric layer that has already standardized the accounting inputs, as opposed to reasoning directly on raw statement rows.

The annual outputs are then persisted in `fundamentals.duckdb`, which becomes the main warehouse used by the downstream reasoning pipeline and the app. Instead of recalculating annual accounting structure from scratch during every run, the system works from a stable warehouse snapshot that already contains standardized statement data and derived metric tables.

The macro side is more exploratory because its source data is less naturally clean. This part of the pipeline is organized under `Common\Macro\Pipeline_DATA`, where the scripts probe the large parquet dump, inspect market and trade schemas, and export the smaller CSV-style values actually used downstream. Before the model dataset is built, a full 40GB parquet dump is downloaded from a source repository containing Federal Open Market Committee Meeting sentiment data. Then, the scripts explicitly probe market and trade schemas, check timestamp availability, inspect coverage, and export filtered event data. In practical terms, this stage reduces the raw source into a much smaller daily table, roughly on the order of about 1,200 rows, spanning all 7 days of the week where needed for macro continuity. Essentially, the macro pipeline does not assume that the source tables are already analysis-ready. Multiple scripts are used first to check whether the underlying market records, trade records, and event fields are reliable enough to support later modelling.

After that probing stage, the macro extraction pipeline builds the time-series dataset used by the XGBoost model. The training script then joins together cleaned SORA daily data, SORA 3M data, SGX REIT index information, and FOMC-related expectation features. It aligns them on a time-safe calendar, shifts rate inputs to point-in-time-safe values, forward-fills where appropriate for calendar consistency, and constructs future SORA targets from the realized path. The resulting dataset therefore becomes a time-aligned forecasting table designed specifically for short-horizon SORA prediction.

Taken together, the data-engineering design is more deliberate than a simple scraping exercise. On the micro side, the task is to convert annual financial statements into a stable warehouse and a derived metric layer. On the macro side, the task is to validate irregular source tables and turn them into a forecasting-ready time series. The final system depends on both: the REIT warehouse provides the annual structural view, while the macro pipeline provides the faster-moving rate-stress view that is later used as a runtime overlay.

4.2.1. Threshold-Based Annual Label Engineering

The annual label in this project is engineered directly from post-filing market behaviour. For each REIT-year, the system takes the fiscal year end as the anchor date, rolls forward to the first available trading day on or after that date, and then compounds abnormal returns over the next 63 and 126 trading days. Abnormal return is defined as the REIT's daily return minus the SGX iEdge REIT index daily return.

The main output is `label_126wd`, which is derived from `car_126wd` and stored together with the anchor date, forward-window end dates, and diagnostic counts in `reit_labels.fact_distress_label`. In practical terms, the label evaluates: after the annual filing anchor, did the REIT materially underperform, roughly track, or clearly outperform the sector benchmark over the following half-year window?

The threshold rule is intentionally simple. As mentioned above, if `car_126wd` < -15%, the REIT is labelled DISTRESSED. If `car_126wd` > +5%, it is labelled HEALTHY. Anything in between is labelled WATCH.

This step is also where the earlier weak-labelling and Snorkel implementation was replaced. Instead of generating proxy labels through a separate weak-supervision model, the project defines its annual target directly from observable post-filing abnormal-return behaviour. The methodology is deliberately return-based rather than price-level-based: it compares REIT returns against the SGX iEdge REIT index, and explicitly ignores `REITN` and `REITR`, so the label reflects relative sector performance rather than raw price movement.

4.3. XGBoost Development

The project did not build only one XGBoost pipeline. It explored three model families, each with a different prediction target and different intended role in the overall system.

P stands for Parquet-direct. This family uses forward-looking macro and rate features taken directly from the large 40GB parquet-based source dump, including FOMC-linked features such as `expected_bps`, to predict rates such as EFFR or SORA. The main scripts for this family are `train_p_1fold_pipeline.py` and `train_p_1fold.py`. This was the most successful family. Even though its data limitations required a conservative 1-fold time-ordered design, its holdout metrics generalized well enough for production use, especially in `run_21` for the `fwd_10_days` and `fwd_15_days` setups.

One implementation clarification is important. Although EFFR was considered during a few earlier design stages, the implemented project standardizes on SORA and does not use EFFR in the final deployed pipeline. The active macro targets, the runtime macro overlay, and the evaluation discussed later in Sections 5.1 and 5.5 all use SORA time series and forward SORA change rather than U.S. policy-rate targets.

A stands for Abnormal Returns. This family attempts to predict a REIT's abnormal performance relative to macroeconomic conditions. Its relevant training script is `train_a_multifold_pipeline.py`. Unlike P, this family had enough pooled rows to support broader panel-style modelling, but it did not generalize well in practice. The main failure mode was ticker memorization: the model repeatedly learned identity shortcuts instead of a stable cross-sectional signal. The resulting evidence was therefore inconclusive rather than production-ready.

R or R* stands for Regime. This family attempts to model macro effects against the SGX iEdge S-REIT index itself, rather than directly forecasting SORA or directly forecasting individual REIT abnormal returns. The relevant scripts are `train_rstar_directional_1fold.py`, `train_rstar_xgboost_walkforward_optuna_deap.py`, and `train_rstar_xgboost_walkforward_optuna_deap_1fold.py`. This family was not used in the final app because its signal remained inconclusive, likely due to a combination of limited data and insufficiently rich features for a robust standalone regime model.

The final production choice was based on iterative experiments on these few different “families” of XGBoost models and data modeling before arriving on a stable result: to “use the P family and reject A and R/R* for runtime use.” Hence why the app’s macro layer is built around rate prediction rather than around abnormal-return prediction or a separate regime classifier.

4.3.1. Multi-Configuration Controlled Experiment Design

The macro experiment was designed as a controlled comparison of different ways to define the prediction task, rather than as a one-shot attempt to fit a single model.

The script varies two main things: the forward horizon and the target formulation. In practice, the tested horizons include 1d, 3d, 5d, 7d, 10d, 14d, 21d, and 63d. The pipeline also supports three prediction targets: `option1_level`, which predicts the future SORA level; `option2_change`, which predicts the signed future change in SORA, meaning whether SORA moves up or down and by how much over the chosen forward window; and `option3_abs_change`, which predicts the absolute size of the future move regardless of direction. These targets are then compared across different short-horizon windows.

Within the successful P family, the project is trying to identify which specific macro target is most reliably forecasted, while still remaining useful for the hybrid REIT-distress system. That is why the comparison is framed around practical usefulness as a runtime overlay.

This is also why shuffle and standard k-fold cross-validation are not appropriate here. The macro dataset is a daily time series, the targets are forward-looking, and the feature set includes lagged path information. If rows were shuffled, or if later periods were allowed to appear in the training folds for earlier validation periods, the model would leak future regime information into its own evaluation.

For the same reason, ordinary k-fold would make the model look stronger than it really is by mixing highly autocorrelated neighbouring periods across train and test. The script therefore uses a strictly time-ordered

split with an explicit gap, so validation is always done on later data that the model was not allowed to see during training.

The final deployed choice is the signed 10-trading-day SORA change target. In script terms, this is the `option2_change` setup. This was more stable and less predisposed to noise. Furthermore, it fits the later hybrid design better than predicting the absolute level of SORA, because the app mainly needs a short-horizon signal of whether refinancing conditions are becoming more or less stressful.

4.3.2. Hyperparameter Search Strategy

The project runs two different search strategies, Optuna and DEAP, against the same time-ordered holdout setup and then lets held-out performance decide the winner. This adversarial design allows the system (and the user) to compare and highlight issues that arise during model training.

For the signed change target, the script is not satisfied with a model that only gets the average error slightly lower. It also checks whether the model gets the direction of the rate move right, because the later distress overlay mainly cares whether refinancing conditions are worsening or easing. That is why the script evaluates directional metrics such as accuracy, F1, and AUC in addition to ordinary regression error. It also rejects weak solutions such as models that predict almost the same value every time or predict one direction too often.

There is a specific reason for this metric choice. One of the clearest lessons from the XGBoost training runs came from the historical P-family `run_19 fwd_10_days` comparison. Under the older F1-first winner logic, Optuna was selected because it had the higher F1 and recall, but DEAP was actually better on AUC, R2, RMSE, and accuracy. In other words, the model that won on F1 was not the stronger model in the broader directional-ranking and fit sense. F1 is threshold-dependent: it depends on turning a continuous prediction into an up-versus-down call at a fixed threshold. AUC is more robust for this purpose because it measures directional ranking quality across thresholds rather than at only one cut point. In practical terms, a model can look better on F1 simply because it calls the positive class more aggressively, yet still be worse on AUC, RMSE, and overall ranking quality. That is why the later model-selection logic moved toward a more careful priority structure instead of treating every F1 gain as decisive. This lesson was then confirmed in `run_20`, where the updated AUC → RMSE → F1 priority switched the 10d winner to DEAP. The archived supporting files are kept in `Miscellaneous\Run_Artifacts_XGBoost\run_19\fwd_10_days` and `Miscellaneous\Run_Artifacts_XGBoost\run_20\fwd_10_days`.

The reason both Optuna and DEAP are used is simple: the project does not want to trust one tuning method blindly. Optuna searches for a strong parameter set in a more direct way, while DEAP searches the same problem from a different angle. DEAP is intentionally kept conservative because the dataset is small, so an overly aggressive search could just fit noise instead of real signal. After both searches finish, the script compares their holdout results on the same test window and keeps the better one.

This adversarial selection logic matters especially because the project had other model families that did not generalize cleanly. The A family had enough pooled rows, but it repeatedly drifted toward ticker memorization rather than a stable cross-sectional signal. The R and R* regime-style family remained inconclusive as well, likely because the feature set and effective sample were still too limited for a strong standalone regime model. In practice, this made the P family the only defensible production candidate.

The historical runs also showed why explicit collapse diagnostics matter. The clearest example came from the A-family `run_26 / fwd_21_days` holdout, where both optimizers produced superficially acceptable F1 scores around 0.717 but were clearly unhealthy in structure: Optuna had `pred_positive_rate = 1.0` with `pred_std_ratio = 0.0189`, while DEAP had `pred_positive_rate = 0.9905` with `pred_std_ratio = 0.0324`. In other words, both models were predicting almost one class only and moving very little, so the headline score overstated their usefulness. Balanced class behaviour is therefore treated only as a sanity check. It does not prove generalization by itself, but it helps reject models that are obviously degenerate before deeper interpretation. A healthier contrast appeared later in A-family `run_28 / fwd_21_days`, where holdout `pred_positive_rate` moved back toward 0.60 and 0.52, with `pred_std_ratio` around 0.51 for both optimizers. The archived evidence

for this comparison is kept in `Miscellaneous\Run_Artifacts_XGBoost\run_26\ fwd_21_days` and `Miscellaneous\Run_Artifacts_XGBoost\run_28\ fwd_21_days`.

One further lesson concerns DEAP specifically. The cleanest example came from the P-family `run_20 / fwd_10_days` comparison, where DEAP surfaced a stronger holdout candidate but did so with a much more aggressive parameter set: `max_depth = 8`, `learning_rate = 0.3`, `min_child_weight = 1.0`, and `reg_lambda = 0.1`. Those settings helped DEAP beat Optuna on AUC, R2, RMSE, and accuracy in that run, but they also illustrated why an unconstrained evolutionary search can drift too easily into overfit-prone corners when the sample is small. The practical response was not to discard DEAP, but to tighten its search space so it would not keep wandering toward very deep, weakly regularized, high-learning-rate solutions. That safeguard is specific to the current small-sample setting and should not be treated as a universal rule: if future datasets become materially larger, those caps may need to be relaxed. The supporting artifacts are archived in `Miscellaneous\Run_Artifacts_XGBoost\run_20\ fwd_10_days`.

4.4. Pipeline and Application Delivery

The delivered system is built around a simple practical idea: do the heavy pipeline work first, persist the results, and let the app read those persisted outputs at runtime. In local development mode, `run_reitteratse1.py` first runs the build pipeline and then launches the Streamlit app. For submission and rerun purposes, the same runtime is copied into `SystemCode`, where `docker-compose.yml` can launch the app directly and `--profile rebuild` can rerun the rebuild path from that folder. The build step refreshes the annual labels, the Mamdani cache, and the rule-trace outputs before the interface opens.

This means the app is not doing the full reasoning pipeline from scratch every time a user clicks a page. Rather, the build stage prepares the warehouse tables that the interface needs, including annual metrics, annual distress labels, annual Mamdani scores, and daily CAR-path rows. The app then resolves the selected simulation date to the latest eligible annual row, the latest eligible macro snapshot, and the latest eligible CAR-path row, and combines them into the runtime `final_distress` score.

This separation between build-time persistence and runtime display makes the system easier to audit, because intermediate outputs such as `fact_distress_label`, `fact_fuzzy_cache`, and `fact_car_path_daily` can be inspected directly. It also makes the dashboard more reproducible, because the user is not depending on live rule induction or raw data recomputation at page-load time.

In submission mode, the repository ships with the committed DuckDB snapshot and the app serves directly from that persisted warehouse by default. In other words, the submitted application is designed to demonstrate the final reasoning outputs reliably, while still preserving a separate rebuild path for development and refresh work.

The execution order for the pipeline first builds abnormal-return labels, then the daily CAR-path table, then the Mamdani input frame with `null_count` and `non_ok_count`, then seeds the Neo4j rule graph, then runs Python Mamdani inference and persists the annual fuzzy outputs, and only after that launches the app and evaluation layers. This sequencing ensures that the runtime dashboard is reading a fully prepared annual base rather than mixing partially rebuilt components.

The design also comes with explicit limits: The test window is still frozen and simulated rather than live, some annual anchors do not yet have a full forward 126-trading-day window so some `label_126wd` values remain NULL, and `non_ok_count` is currently persisted mainly for diagnostics rather than used directly in scoring.

5. Findings and Discussion

5.1. Evaluation for XGBoost Best Version

The best locally evidenced macro model is the P-family `run_21 fwd_10_days option2_change` model, where Optuna is the recorded winner over DEAP. Here, `option2_change` means the model predicts the signed 10-trading-day forward change in SORA rather than the future SORA level itself. This is the exact target family wired into the runtime app.

Its holdout summary is:

- $R^2 = 0.1927$
- $RMSE = 0.2528$
- $MAE = 0.2086$
- $Accuracy = 0.6795$
- $Precision = 0.5926$
- $Recall = 0.7385$
- $F1 = 0.6575$
- $AUC = 0.7341$

These are not “perfect prediction” numbers, but they are strong enough to justify using the macro model as a short-horizon overlay. The clearest evidence comes from the deployed P-family `run_21` results for `fwd_10_days` and `fwd_15_days`. In both cases, holdout R^2 stayed only around 0.19–0.20, which means the model is not an especially precise point forecaster. However, the directional metrics were clearly better: AUC reached about 0.73–0.77, accuracy about 0.68–0.72, and F1 about 0.64–0.69.

Just as importantly, `run_21` also looked structurally healthier than weaker historical runs. The winning `fwd_10_days` setup kept `pred_positive_rate` = 0.5192 with `pred_std` = 0.1770, while the `fwd_15_days` Optuna winner kept `pred_positive_rate` = 0.4551 with `pred_std` = 0.1039. The model was neither predicting one direction almost all the time, nor was it collapsing toward near-constant outputs. This exhibits materially healthier than the degenerate prediction patterns seen in weaker historical runs.

Hence, the practical reason the XGBoost layer is used the way it is. It is better at telling the system whether near-term rate stress is worsening or easing than at forecasting the exact future SORA level. The annual Mamdani layer therefore remains the structural core, while XGBoost nudges the final score when the short-horizon rate backdrop becomes more adverse or more supportive. The archived supporting files are kept in `Miscellaneous\Run_Artifacts_XGBoost\run_21\fwd_10_days` and `Miscellaneous\Run_Artifacts_XGBoost\run_21\fwd_15_days`.

5.2. Evaluation for XGBoost Historical Versions

The historical runs are most useful when read by model family rather than by run number alone. The P family was the only one that produced a stable enough directional signal to justify deployment, even under a conservative 1-fold time-ordered setup. By contrast, the A abnormal-returns family often looked interesting in isolated metrics but did not generalize cleanly, while the R and R* regime-style family remained too inconclusive for production use. This is the practical reason the final app uses only P.

Three recurring lessons came out of those historical runs. First, winner selection needed to be stricter than an F1-first rule. The clearest example is P-family `run_19 / fwd_10_days`, where Optuna won on F1 and recall but DEAP was better on AUC, R^2 , RMSE, and accuracy. `Run_20 / fwd_10_days` then confirmed the fix by switching the winner after the priority was tightened. Second, collapse diagnostics mattered because some models looked acceptable on a headline score while becoming structurally unusable. The clearest case is A-family `run_26 / fwd_21_days`, where both optimizers had F1 near 0.717 but also predicted almost one class only and showed extremely weak output dispersion. Third, ticker identity could contaminate the A family. `Run_22` still contained explicit ticker one-hot features, and the SHAP and holdout exports show the model relying on those identity signals directly. Later `run_27` and `run_28` variants removed ticker-identity features and treated cleaner universes as a diagnostic check, which is more consistent with learning a transferable state-based relationship than memorizing specific names. The archived evidence for these comparisons is in `Miscellaneous\Run_Artifacts_XGBoost\run_19\fwd_10_days`, `run_20\fwd_10_days`, `run_22\fwd_10_days`, `run_22\fwd_15_days`, `run_26\fwd_21_days`, `run_27\fwd_21_days`, and `run_28\fwd_21_days`.

5.3. Evaluation for Mamdani Layer and Full Pipeline

The current final evaluation output is `Common\Eval\IO\run_3`, which compares the simple baseline, the annual Mamdani layer, the refinancing-only layer, and the full hybrid score on the same evaluation slice. A

copy of the relevant outputs is archived in `Miscellaneous\full_pipeline_eval\run_3`.

The summary metrics show:

Model	Label Accuracy	Macro F1	MCC	Continuous MAE	Continuous RMSE
distress_baseline	0.3572	0.3553	0.0538	0.3534	0.4747
distress_score_mamdani	0.5563	0.5294	0.2969	0.3100	0.3635
distress_score_refi	0.2816	0.2817	0.2261	0.3638	0.5175
final_distress	0.5214	0.5188	0.2919	0.2724	0.3295

The main result is a trade-off between cleaner annual classification and better continuous ranking. The Mamdani annual layer has the best label accuracy at 0.5563, which means it is strongest if the goal is to reproduce the annual class label directly. The final hybrid score is slightly worse on label accuracy at 0.5214, but it has the best continuous-error profile, with MAE = 0.2724 and RMSE = 0.3295, both better than Mamdani alone. That pattern is consistent with the design. The hybrid layer is not meant only to restate the annual label. It is meant to behave as a smoother runtime risk score after adding macro-rate and CAR-path information.

The class-level results make the trade-off even clearer. Mamdani gives a more balanced treatment of the WATCH bucket, with WATCH recall of 0.6826, while the final hybrid score is more aggressive toward distress: DISTRESSED recall rises from 0.6841 under Mamdani to 0.8545 under the full model. This comes at a cost, because more borderline WATCH names get pulled upward into DISTRESSED. The refinancing-only layer performs worst as a standalone classifier, which is also expected. It was never designed to be a complete distress model on its own; it is only one stress channel inside the final synthesis.

The ranking metrics support the same interpretation. `Final_distress` has the best MAP@5 at 0.7274, ahead of Mamdani at 0.7131, which suggests the hybrid score is slightly better at surfacing the riskiest names near the top of a ranking even when its final class labels are less conservative. In practical terms, Mamdani is the cleaner annual classifier, while the full pipeline is the better runtime prioritization score. The only caveat is that the final hybrid can be quite aggressive, with a precision score close to or below 50%.

5.4. Representation Tables and Graphs

The report already contains several directly usable supporting artifacts. For XGBoost, the most useful artifacts are the `run_21` holdout summaries and the historical SHAP visuals now archived under `Miscellaneous\Run_Artifacts_XGBoost\run_21`, `run_22`, `run_27`, and `run_28`. For the full pipeline, the most useful artifacts are the summary, per-class, ranking, and confusion-matrix CSVs archived under `Miscellaneous\full_pipeline_eval\run_3`.

The confusion-matrix outputs already show the core behavioral difference between Mamdani and the full hybrid score. Mamdani correctly identifies 1,594 distressed rows and 4,811 watch rows. The final hybrid score identifies more distressed rows correctly at 1,991, but it does so by reclassifying more WATCH rows upward into DISTRESSED (2,264 such cases, versus 1,698 for Mamdani). This is why the hybrid model looks more aggressive in practice. It improves distressed-case capture, but it also increases pressure on borderline names.

The existing SHAP artifacts serve a different purpose. They are most useful in the historical-model discussion because they help show whether the model is learning macro and path features or relying too heavily on ticker identity. In other words, the tables explain performance, while the SHAP graphs help explain why some historical variants were accepted or rejected.

5.5. Optuna and DEAP as an Adversarial Error-Surfacing System

The most useful way to describe the macro training pipeline is that Optuna and DEAP were not used only to search for a better score. They were used to expose weaknesses in each other. The script makes that design explicit. In `train_p_1fold_pipeline.py`, the final winner is not picked by whichever optimizer finishes last or whichever has the most impressive single metric. It goes through a formal `choose_winner(...)` rule with tolerances, where signed targets are judged first on AUC, then on RMSE, and only then on F1. That is a direct response to the historical `run_19` and `run_20` lesson that a model can win on thresholded F1 while still being weaker on broader directional ranking quality. A copy of the relevant script is archived in `Miscellaneous\script_refs\train_p_1fold_pipeline.py`.

The script also shows that the final search regime was deliberately conservative. Optuna is fixed at 80 trials. DEAP is limited to 8 generations with population size 20, mutation probability 1/9, crossover probability 0.6, and tournament size 3. More importantly, the script explicitly labels its DEAP grid as a “conservative DEAP search space for the current small-row regime.” In practice, that means the evolutionary search is no longer allowed to roam freely across the same aggressive corners seen in earlier runs. The later search space narrows tree depth to 2–5, keeps learning rates moderate, and raises the floor on `min_child_weight` and `reg_lambda` relative to the earlier aggressive `run_19` and `run_20` candidates. This is not because DEAP is inherently worse than Optuna. It is because, in a small temporal dataset, an unconstrained genetic search can optimize noise very convincingly. The historical runs, the final script, and the tightened parameter ranges therefore all point to the same conclusion: the optimizer contest became an adversarial error-surfacing system, but only after the search itself was disciplined enough not to reward unstable models. The supporting script snapshots are archived in `Miscellaneous\script_refs`, and the relevant historical runs remain archived under `run_19`, `run_20`, and `run_21`.

5.6. Developed Models and Final Interpretation

The final system should be read as four distinct views of distress rather than as one model with minor variations. `Distress_baseline` is the simplest benchmark and mainly shows how weak a naive annual mapping would be on its own. `Distress_score_mamdani` is the annual structural reading built from financial ratios and rule combinations. `Distress_score_refi` isolates refinancing stress as one channel only. `Final_distress` is the runtime synthesis that blends annual structure, refinancing sensitivity, macro stress, and CAR-path information.

The evaluation makes the role of each layer fairly clear. Mamdani is the strongest standalone annual classifier. It has the best label accuracy and keeps a more balanced treatment of the WATCH bucket, which is why it should be understood as the system’s main annual reasoning core. The refinancing-only score performs poorly as a full classifier, but that is not a failure of design. It was never meant to replace the annual layer; it is a narrow stress amplifier. The full hybrid score is the most operationally aggressive view. Its distressed recall rises to 0.8545, compared with 0.6841 for Mamdani alone, which means it catches more distressed cases. The price is lower precision and more upward reclassification of borderline names.

That trade-off is exactly what the architecture is trying to achieve. The deployed system is not claiming that the macro model or CAR path should override annual fundamentals by themselves. Instead, the annual Mamdani score supplies the stable structural anchor, while the macro and market overlays make the live score more sensitive when conditions deteriorate between annual reporting points. In practical terms, Mamdani is the more stable annual interpretation, while `final_distress` is the more useful monitoring score when the goal is to surface names that may be drifting into trouble before the next full-year financial picture is available.

This also clarifies why only P was promoted from the XGBoost experiments. The macro model has enough signal to improve the runtime overlay, but not enough to replace the reasoning system. The A and R/R* families were not rejected because they were useless in every respect; they were rejected because they did not produce a stable enough signal for the production role the app needed. The archived evaluation outputs for this interpretation are in `Miscellaneous\full_pipeline_eval\run_3`, and the historical macro evidence remains archived under the relevant `run_19` to `run_28` folders.

6. Future Work

The proposal and implementation checklist together suggest a coherent future-work agenda.

First, the current repository is intentionally built on frozen and simulation-resolved data rather than on a live production feed. That is a good choice for reproducibility, but it also means the system is still a proof of concept. A production version would need:

- reliable live ingestion for both firm-level and macro data
- scheduled warehouse refresh
- automated cache rebuild orchestration
- stronger monitoring around failed upstream refreshes

Second, the current label scheme and final hybrid weights are explicitly not final. The local checklist still marks several items as unfinished:

- threshold tuning beyond the current -15% / $+5\%$ scheme
- deeper Mamdani calibration
- possible inclusion of `non_ok_count` in the scoring logic
- macro and CAR-path weighting refinement

Third, the app itself is functional but still has room for a more mature front end:

- browser-level QA
- richer rule-firing visualization
- closer visual alignment with the design artifacts
- better responsive handling

Fourth, the proposal's broader ambitions remain valid:

- expand coverage to more REITs or adjacent dividend vehicles
- build a more formal MLOps pipeline
- test additional macro experts or macro targets
- extend the system into a more general explainable investment-risk framework

On the macro-model side specifically, future work should revisit the current conservative DEAP guardrails once materially larger datasets are available. Those restrictions are sensible for the present small-row regime, but they should not be treated as permanent if later versions of the project gain enough data to support a broader and more expressive search space.

6.1. Conclusion and Afterthoughts

Two design questions remain especially worth reflecting on after the final evaluation. First, sectoral differences should not be assumed to improve this project at the current universe size. The historical XGBoost work already showed that once ticker identity was included, the model started memorizing tickers rather than learning a transferable signal. With a small N universe, a sector feature could create a similar shortcut problem if a few distressed names dominate the label pattern inside one sector. In this project, names such as `D5IU` and `0XMU` can easily distort that balance. For that reason, sector is more likely to become useful only when the REIT universe is materially larger. At the current scale, adding sector directly could improve in-sample separation while actually worsening generalization.

Second, the project already has a smoothing mechanism, and it comes from the move away from the original Snorkel design toward Mamdani fuzzy rule induction. The annual Mamdani score is already the main smoothing layer because it turns noisy raw ratios into a graded annual anchor instead of a brittle point prediction. On top of that, the code already damps weak-evidence cases back toward neutral, and the CAR-path overlay already has a dead zone so very small market moves do not immediately change the runtime score. This can be demonstrated by observing the code and its effects in `Common\Micro\5_Model_KG\reitteratsel_core.py`. Specifically, the code section with modifiers for the `macro_shock` and `car_path_shock` in `compute_final_distress_score()` means short-horizon fluctuations

can adjust the risk reading, but they are still damped by the stronger annual rule-based anchor. Hence, Mamdani is already doing the main smoothing, and the hybrid overlay design already includes extra damping against overreaction.

7. References

SystemCode (runnable app)

- SystemCode\docker-compose.yml
- SystemCode\docker-compose.env
- SystemCode\Frontend\Dockerfile.reitteratsel
- SystemCode\Frontend\reitteratsel_app.py
- SystemCode\Frontend\reitteratsel_view_logic.py
- SystemCode\Micro\5_Model_KG\build_reitteratsel_pipeline.py
- SystemCode\Micro\5_Model_KG\reitteratsel_core.py
- SystemCode\Micro\5_Model_KG\mamdani_rule_seed.json
- SystemCode\Micro\IO\out_annual_warehouse\fundamentals.duckdb
- SystemCode\Macro\IO\Model_Train\Use\run_21

Common (development forks, data processing, and engineering artifacts)

- Project map and implementation notes: Common\PROJECT_REFERENCE_MAP.md Common\Micro\5_Model_KG\DesignDocs Common\Micro\5_Model_KG\DesignDocs\Implementation_Checklist_v1a.md
- Micro scraping / HTML export / serialization references: Common\Micro\1_TradingView_Exploration Common\Micro\2_HTML_Dumping Common\Micro\3_Serialize_Dump_To_CSV_Parquet
- Micro / warehouse / rule-engine references: Common\Micro\4_Compute_Metrics\Data_Dict_Reit_Metrics.md Common\Micro\4_Compute_Metrics\Schemas.md Common\Micro\5_Model_KG\mamdani_rule_seed.json Common\Micro\5_Model_KG\reitteratsel_core.py
- Macro data-probing / extraction references: Common\Macro\Pipeline_DATA
- Macro training-script references: Common\Macro\Pipeline_MODEL\5_XGBoost\train_p_1fold_pipeline.py Common\Macro\Pipeline_MODEL\5_XGBoost\train_a_multifold_pipeline.py Common\Macro\Pipeline_MODEL\5_XGBoost\train_rstar_xgboost_walkforward_optuna_deap.py Common\Macro\Pipeline_MODEL\5_XGBoost\train_rstar_xgboost_walkforward_optuna_deap_1fold.py
- Frontend and repo-level references: Common\Frontend\DesignDoc\Reitteratsel.pdf README.md

Miscellaneous (key development artifacts)

- Miscellaneous\Run_Artifacts_XGBoost\run_19\fwd_10_days
- Miscellaneous\Run_Artifacts_XGBoost\run_20\fwd_10_days
- Miscellaneous\Run_Artifacts_XGBoost\run_21\fwd_10_days
- Miscellaneous\Run_Artifacts_XGBoost\run_21\fwd_15_days
- Miscellaneous\Run_Artifacts_XGBoost\run_22\fwd_10_days
- Miscellaneous\Run_Artifacts_XGBoost\run_22\fwd_15_days
- Miscellaneous\Run_Artifacts_XGBoost\run_26\fwd_21_days
- Miscellaneous\Run_Artifacts_XGBoost\run_27\fwd_21_days
- Miscellaneous\Run_Artifacts_XGBoost\run_28\fwd_21_days
- Miscellaneous\full_pipeline_eval\run_3
- Miscellaneous\script_refs\train_p_1fold_pipeline.py
- Miscellaneous\script_refs\train_a_multifold_pipeline.py

External literature

- [1] T. Shumway, "Forecasting Bankruptcy More Accurately: A Simple Hazard Model," *The Journal of Business*, vol. 74, no. 1, pp. 101-124, 2001, doi: 10.1086/209665.

- [2] J. Y. Campbell, J. Hilscher, and J. Szilagyi, "In Search of Distress Risk," *NBER Working Paper* no. 12362, 2006.
- [3] W.-Y. Cheng, E. Su, and S.-J. Li, "A financial distress pre-warning study by fuzzy regression model of TSE-listed companies," *Asian Academy of Management Journal of Accounting and Finance*, vol. 2, no. 2, pp. 75-93, 2006.
- [4] N. V. Martyushev, V. Spitsin, R. V. Klyuev, L. Spitsina, V. Yu. Konyukhov, T. A. Oparina, and A. E. Boltrushevich, "Predicting firm's performance based on panel data: Using hybrid methods to improve forecast accuracy," *Mathematics*, vol. 13, no. 8, p. 1247, 2025, doi: 10.3390/math13081247.
- [5] A. Ratner, S. H. Bach, H. Ehrenberg, J. Fries, S. Wu, and C. Re, *Snorkel: Rapid Training Data Creation with Weak Supervision*. Stanford, CA, USA: Stanford AI Lab, 2017.
- [6] BDO Singapore, *REIT Leverage and Disclosure*. Singapore: BDO Singapore, 2025.

Appendix A. Project Proposal

The previous proposal is attached in this folder as `ProjectReport\PROPOSAL_Group16_JasonTay_REITerratsel.pdf`.

- Project title: REITerratsel - Equity Risk Solver for S-REITs
- Core problem: transform fragmented S-REIT financial and macro data into an interpretable distress-ranking workflow.
- Original technique mix: knowledge-based reasoning, weak supervision, rule extraction, Neo4j knowledge graph, and Streamlit interface.
- Key change since proposal: the final implementation replaced Snorkel-centric labelling with threshold-based label engineering from cumulative abnormal returns.

Appendix B. Mapped System Functionalities against MR, RS, CGS Modules

The project satisfies the requirement to integrate at least three IRS-related technique groups. More importantly, these are not abstract labels added after the fact. Each module mapping corresponds directly to parts of the implemented system already discussed in Sections 3, 4, and 5.

Appendix B.1. Decision Automation

The Mamdani fuzzy layer is the clearest decision-automation component. As explained in Sections 3.4 and 3.5, it turns annual accounting and balance-sheet conditions into a machine-executable distress score through explicit rules rather than through free-form analyst judgment. The rules do not merely display ratios. They automate decisions such as when weak coverage, high leverage, payout strain, and refinancing pressure should reinforce one another and escalate a REIT's risk reading. Section 5.3 then shows that this annual reasoning layer is the strongest standalone classifier in the system, which is why it is appropriate to treat it as the main decision-automation core.

Appendix B.2. Business Resource Optimization / Evolutionary Computing

The XGBoost macro pipeline satisfies the optimization or evolutionary-computing requirement through its Optuna and DEAP search layers. Sections 4.3.2 and 5.5 show that these are not decorative additions. They directly affect which macro model is accepted into the runtime overlay, how unstable candidates are filtered out, and how aggressive search behavior is constrained in small-sample settings. In other words, the optimization component has a causal role in the final design: without it, the deployed P-family macro overlay and its winner-selection logic would not have been chosen as they were.

Appendix B.3. Knowledge Discovery and Data Mining

The project also clearly qualifies as knowledge discovery and data mining. Sections 3.2, 3.3, and 4.2-4.4 show that the system does not start from ready-made labels or tidy model tables. It builds them. On the micro side, annual financial statements are transformed into distress-relevant metrics and status flags. On

the macro side, market and rate data are converted into lag, momentum, curve, and forward-signal features. The ground truth itself is engineered from cumulative abnormal-return behavior rather than copied from an external labelled dataset. This is a causal part of the project design: the later Mamdani and XGBoost layers only exist because earlier data-mining stages turned heterogeneous raw inputs into model-ready and rule-ready structures.

Appendix B.4. Cognitive Techniques / Tools

The cognitive-techniques component is most visible in the project's use of explicit knowledge representation. As described in Sections 3.4, 3.5, and 4.4, the Mamdani rules are seeded into a Neo4j rule graph during rebuild and then persisted into a form the runtime app can use through DuckDB. Even though the app does not query Neo4j live on every interaction, the system still relies on a graph-backed representation of domain knowledge during construction of the reasoning layer. That creates a clear causal link between symbolic knowledge representation and the final distress outputs: the fuzzy rules are not just coded as opaque weights, but organized as explicit, inspectable business logic that can be traced through rule IDs and rule explanations in the final interface.

Appendix C. Installation and User Guide

For full run instructions, please refer to README.md, Section 5: User Guide.

An abridged version for submission and demonstration use is included below.

C.1. Docker submission / demo mode

Pass:

```
cd <path-to-this-repo>
cd SystemCode
```

App-only mode This serves the app against the committed DuckDB snapshot and does not need `docker-compose.env`.

```
docker compose up --build
```

Then open:

```
http://localhost:8501
```

To stop it:

```
docker compose down
```

Appendix D. Data Sources

The implemented system relies on the following main external data sources.

- SORA time series (Monetary Authority of Singapore): <https://eservices.mas.gov.sg/statistics/dir/DomesticInt>
- Time-series market data and financial-statement data (TradingView): <https://www.tradingview.com/>
- Bulk 40GB data dump (including FOMC sentiment dump) from Jonathan Becker (Jon-Becker on GitHub): <https://s3.jbecker.dev/data.tar.zst>